

is_clamped

Abstract

Introduce `std::is_clamped(v, lo, hi)` to mean `lo <= v && v <= hi`, except that `v` is only evaluated once.

Now	Proposed alternative
<code>Axis::x <= Coord::axis && Coord::axis <= Axis::z</code>	<code>std::is_clamped(Coord::axis, Axis::x, Axis::z)</code>

Now	Proposed alternative + P0330 [4]
<code>1u <= digits.size() && digits.size() <= 7u</code>	<code>std::is_clamped(digits.size(), 1uz, 7uz)</code>

Revision history

- `cv void` -> R0
 - 2019y/January/19, pre-Kona.

Problem

Expressions equivalent to `lo <= v && v <= hi` are common. They are used to check whether `v` is within the range `[lo, hi]`. However, because `v` appears twice, they

- are error prone to type and update,
- reduce readability as `v` gets longer, and
- may degrade performance due to the repeated evaluation of `v`.

We propose `std::is_clamped(v, lo, hi)` as an alternative spelling that better captures intent and is potentially more performant.

Alternatives

Chaining Comparisons (rejected)

P0893 [1] would have allowed us to write `lo <= v <= hi`. Its rejection made us write this proposal.

There was no consensus on making comparison operators chain, nor was there encouragement for further work in that area.
-- P1018R2 [2]

Initialize intermediate variable with `v`

It may be undesirable for some of the reasons selection and iteration statements have been getting space for initialization.

In some contexts it is impossible, such as member initializer lists and requires clauses.

Immediately invoked lambda expression

This works, at the cost of much syntactical noise on par with repeating `v` :

- `[&] { const auto& w{v}; return what we actually care about ; }()` , or
- `[] (const auto& v) { return what we actually care about ; }(v)` .

Safe integral comparisons' `std::in_range`

Vicente J. Botet Escriba made us aware [6] of `std::in_range` proposed in P0586 [7]. Its name suggests similarity to our proposed `std::is_clamped` . However, it checks whether an integral value is in the range of an integral type with safe integral comparisons proposed in the same paper. Neither can subsume the other.

Bikeshedding

The following have been suggested of the proposed `std::is_clamped` design:

- Different names.
- Allowing arguments of different types.
- Marking a bound as closed or half-open.

However, we believe the proposed design was fixed once ISO/IEC 14882:2017 (C++17) was published with `std::clamp` . After all, we mean for `std::is_clamped` to complement `std::clamp` in the same way `std::is_sorted` complements `std::sort` .

Wording

Add the following declarations to [algorithm.syn] after those of `clamp` .

```
template<class T>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Compare comp);

namespace ranges {
    template<class T, class Proj = identity,
            IndirectStrictWeakOrder<projected<const T*, Proj>> Comp = ranges::less<>>
            constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Comp comp = {}, Proj proj = {});
}
```

Add the following detailed specifications to [alg.clamp] after those of `clamp` .

```
template<class T>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Compare comp);

namespace ranges {
    template<class T, class Proj = identity,
            IndirectStrictWeakOrder<projected<const T*, Proj>> Comp = ranges::less<>>
            constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Comp comp = {}, Proj proj = {});
}
```

Expects: The value of `lo` is no greater than `hi` . For the first form, type `T` shall be *Cpp17LessThanComparable* (Table 24).

Returns: `true` if

- `v` is not less than `lo`, and
 - `hi` is not less than `v`.
- Otherwise, `false`.

[Note: If NaN is avoided, `T` can be a floating-point type. -- end note]

Complexity: At most two comparisons and three applications of the projection, if any.

Possible implementation

```
template<class T>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi)
{
    return !(v < lo) && !(hi < v);
}

template<class T, class Compare>
constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Compare comp)
{
    return !comp(v, lo) && !comp(hi, v);
}

namespace ranges {
    template<class T, class Proj = identity,
            IndirectStrictWeakOrder<projected<const T*, Proj>> Comp = ranges::less<>>
    constexpr bool is_clamped(const T& v, const T& lo, const T& hi, Comp comp = {}, Proj proj = {})
    {
        auto&& pv{INVOKE(proj, v)};
        return !INVOKE(comp, pv, INVOKE(proj, lo)) &&
            !INVOKE(comp, INVOKE(proj, hi), pv);
    }
}
```

Related proposals

Paper	Title	Relation
P1243 [3]	Rangify New Algorithms	Introduces <code>std::ranges::clamp</code> .

Acknowledgements

Thanks to Ray Hamel, Vicente J. Botet Escriba, and everyone else for their comments in the std-proposals thread [5].

References

- [1] Chaining Comparisons, <https://wg21.link/P0893>
- [2] Evolution status after San Diego 2018, <https://wg21.link/P1018r2>
- [3] Rangify New Algorithms, <https://wg21.link/P1243>
- [4] Literal Suffixes for `ptrdiff_t` and `size_t`, <https://wg21.link/P0330>
- [5] `is_clamped`, <https://groups.google.com/a/isocpp.org/forum/?fromgroups#!topic/std-proposals/LDhzb-58sV4>

[6] Personal communications

[7] Safe integral comparisons, <https://wg21.link/P0586>